



Universidad
Manuela Beltrán
Acreditada en Alta Calidad Multicampus

Ingeniería web II

María del Carmen Peña Suarez

Juan David Gomez Rojas

Cristhian Felipe Ortiz Monter

Universidad Manuela Beltrán

PROYECTO

Bogotá, Colombia

2026

FullBike — Blueprint del Proyecto

Resumen ejecutivo

FullBike es un **ERP interno para un negocio de motos y repuestos en Colombia** (a pesar del nombre, no es un sistema de alquiler de bicicletas). Cubre el ciclo completo de la operación: clientes, empleados, proveedores, inventario, ventas con impuestos y descuentos, y nómina con normativa laboral colombiana.

El proyecto nació como una aplicación web monolítica (ASP.NET MVC, sin API ni integraciones externas). Este blueprint documenta tanto la base original como la **API REST interna y tres integraciones externas** (facturación electrónica DIAN, pasarela de pagos Wompi, notificaciones por email) agregadas para que el sistema pueda conectarse con otros canales (app móvil, punto de venta, portal de cliente) y automatizar procesos hoy manuales.

Stack tecnológico

Capa	Tecnología
Framework web	ASP.NET MVC 5 (.NET Framework 4.7.2), vistas Razor
API REST	ASP.NET Web API 2 (System.Web.Http), agregada sobre el mismo proyecto
ORM	Entity Framework 6.5.1, Code First (migraciones presentes pero desactivadas en runtime)
Base de datos	SQL Server LocalDB (ERP_Database)
Autenticación	Forms Authentication (cookie) para el sitio; API key por header para clientes externos
Front-end	Bootstrap 5.2.3, jQuery 3.7.0, sin framework SPA
Paquetes clave	Newtonsoft.Json, Microsoft.AspNet.WebApi.*, Microsoft.AspNet.Cors
Gestión de paquetes	NuGet clásico (packages.config), no SDK-style/PackageReference

Es un proyecto **legacy pero estándar**: no usa contenedores, no tiene pipeline de CI/CD, y el despliegue es manual (IIS/IIS Express).

Arquitectura

No hay capa de servicios ni repositorios: cada Controller MVC accede directo a `ERPContext` (`private ERPContext db = new ERPContext();`). La única excepción es la clase estática `CalculadoraVentas` para cálculos de IVA/retención. La API y las integraciones agregadas introducen las primeras capas de `Services/` del proyecto.

```
FullBike/
├── App_Start/           Configuración de rutas MVC y Web API
├── Controllers/        Controllers MVC (vistas Razor)
│   └── Api/            Controllers Web API (ApiController) – agregados
├── Models/             Entidades EF + ERPContext + CalculadoraVentas
├── Services/           Capa de servicios – agregada
│   ├── Dian/           Facturación electrónica (Factus)
│   ├── Pagos/          Pasarela de pagos (Wompi)
│   └── Notificaciones/ Envío de email (SMTP)
├── Views/              Vistas Razor por módulo
├── Migrations/         Migraciones EF (desactivadas) + script SQL manual
└── Web.config           Conexión a BD, appSettings de todas las integraciones
```

La autorización hoy es inconsistente: `[Authorize]` protege Ventas, Descuentos y Dashboard, pero **Cientes, Empleados, Proveedores, Repuestos y Nómina quedan abiertos** a cualquiera con la URL (pendiente en la sección de seguridad).

Módulos de negocio

Módulo	Qué hace
Cientes	CRUD de clientes, borrado lógico (<code>Activo=false</code>), tipo Regular/Premium/Corporativo
Empleados	CRUD, login/rol, contrato, seguridad social, liquidación al retiro
Proveedores	CRUD de proveedores de repuestos
Inventario (Repuestos)	Motos nuevas/usadas, repuestos y accesorios, con stock y precio
Ventas	Carrito multi-producto, IVA 19%, retención en la fuente por tramos, descuentos, descuento automático de stock, factura automática, abonos parciales
Descuentos	Reglas configurables por cliente/categoría/monto mínimo/vigencia
Nómina	Pago de nómina y liquidación de contrato con aportes de ley (salud, pensión, ARL, caja de compensación, ICBF, SENA)

Todos los cálculos de impuestos y aportes siguen normativa colombiana vigente en el código (no se validan contra un servicio externo, son fórmulas fijas).

Modelo de datos

Entidades principales (EF Code First, sin pluralizar nombres de tabla):

flowchart LR

```
Cliente --> Venta
Empleado --> Venta
Venta --> DetalleVenta
Repuesto --> DetalleVenta
Venta --> Factura
Venta --> AbonoCliente
Descuento --> DescuentoAplicado
Venta --> DescuentoAplicado
Proveedor --> Repuesto
Empleado --> PagoNomina
Empleado --> LiquidacionContrato
```

- **Venta** es el centro del modelo: referencia Cliente y Empleado, y produce DetalleVenta (líneas), Factura (incluye ahora CufeDian/EstadoDian/FechaEnvioDian) y AbonoCliente (pagos parciales, incluidos los que llegan por Wompi).
- **AuditoriaVenta** existía sin usarse; ahora registra fallos de las integraciones DIAN y email.
- **CalculadoraVentas** centraliza IVA (19%) y retención en la fuente (2.5% > \$1M, 3.5% > \$5M) — la usan tanto el flujo MVC como la API.

API REST interna

Autenticación: cookie de sesión (llamadas AJAX del propio sitio) o header X-Api-Key (clientes externos), validados ambos por [Authorize].

Endpoint	Método	Qué hace
/api/clientes	GET	Lista clientes
/api/clientes/{id}	GET	Detalle de cliente
/api/clientes/{id}/saldo	GET	Saldo pendiente (ventas – abonos)
/api/repuestos	GET	Lista repuestos (filtros activos, conStock)
/api/repuestos/{id}	GET	Detalle de repuesto
/api/ventas	GET	Lista ventas
/api/ventas/{id}	GET	Detalle de venta con líneas
/api/ventas	POST	Crea una venta (misma lógica de impuestos/stock que el flujo

Endpoint	Método	Qué hace
		MVC)
/api/facturas/{id}/reenviar-dian	POST	Reintenta el envío a Factus
/api/pagos/generar-link	POST	Genera el link de checkout de Wompi
/api/pagos/webhook	POST	Recibe confirmación de pago de Wompi (anónimo, validado por firma)
/api/notificaciones/reenviar-factura/{id}	POST	Reenvía el email de una factura

Abre la puerta a un futuro cliente móvil, un punto de venta físico o un portal de autoservicio para clientes, sin tocar las vistas MVC existentes.

Integraciones externas

Las tres quedan cableadas end-to-end pero **desactivadas por configuración** (Enabled=false en Web.config) hasta que exista una cuenta real — evita romper el flujo de ventas mientras no hay credenciales.

Integración	Proveedor elegido	Disparador	Estado
Facturación electrónica DIAN	Factus	Al generar cada factura en VentasController.Create	Deshabilitada, lista para activar
Pasarela de pagos	Wompi	Botón de pago / webhook de confirmación	Deshabilitada, lista para activar
Notificaciones	SMTP estándar	Confirmación de venta y de pago recibido	Deshabilitada, lista para activar

Si una integración falla (ej. Factus caído), **la venta se completa igual**: el error queda registrado en la tabla AuditoriaVenta (antes sin uso) en vez de bloquear al usuario.

Seguridad y deuda técnica conocida

- Passwords de empleados hasheadas con **MD5** (inseguro para producción; el propio código lo señala en un comentario).
- Login de HomeController tiene credenciales **hardcodeadas** (admin/admin123) como placeholder.
- [Authorize] no está aplicado de forma consistente: Clientes, Empleados, Proveedores, Repuestos y Nómina son accesibles sin sesión.

- La llave `Api:Key` en `Web.config` trae un valor de ejemplo (CAMBIAR-ESTA-LLAVE-EN-PRODUCCION) que **debe reemplazarse** antes de exponer la API fuera de la red local.
- Las migraciones EF están desactivadas en runtime (`Database.SetInitializer<ERPContext>(null)`); los cambios de esquema (incluido el de este blueprint) se aplican con scripts SQL manuales.

Próximos pasos recomendados

1. Correr `Migrations/Sql/AddDianFieldsToFactura.sql` contra la base de datos.
2. Crear las cuentas de Factus (DIAN) y Wompi, y activarlas en `Web.config` (`Enabled=true`) cuando haya credenciales.
3. Configurar un SMTP real (host/puerto/credenciales) para activar las notificaciones por email.
4. Cerrar el hueco de autorización: agregar `[Authorize]` a los controllers que hoy quedan abiertos.
5. Reemplazar MD5 por un hash moderno (bcrypt/PBKDF2) para las contraseñas de empleados.
6. Cambiar la `Api:Key` de ejemplo por un valor real antes de exponer la API fuera de la red local.